



Mini Event-Planer

Modul 295

NovaSphere Events GmbH
11. Juli 2026

«Where Moments Become Stars.»



Projektdokumentation

Projekttitel: Mini Event-Planer
Projektziel: Das Ziel des Projekts **Mini Event-Planer** (Modul 295) ist die Entwicklung eines stabilen Backends, mit dem kleine Events effizient erstellt, verwaltet und organisiert werden können. Die REST-API erlaubt es, Veranstaltungen mit allen relevanten Informationen wie Titel, Datum, Location und Artist zu erfassen und persistent in einer PostgreSQL-Datenbank zu speichern.
Der Fokus liegt auf einer sauberen 3-Schicht-Architektur, DTO-Mapping, zuverlässiger Validierung und Fehlerbehandlung sowie automatisierten Tests. Das Projekt verfolgt das Ziel, eine leicht verständliche, moderne und technisch saubere Lösung zu liefern, die als professionelles Produkt der **NovaSphere Event GmbH** präsentiert werden kann.
Projektstart: 4. Juli 2026
Projektende: 11. Juli 2026
Auftraggeber: Graziano Laveder
Projektleiter: Luca Caputi
Teammitglieder: Marco Fritsche, Luca Caputi
Versionsnummer: 1.2.0

Version	Datum	Autor	Änderung
1.0.0	08.07.2026	Luca Caputi	Erstausgabe
1.1	09.07.2026	Luca Caputi	Anpassung und Fehlerkorrektur
1.1.5	10.07.2026	Luca Caputi	Abgleich und Anpassung
1.2.0	11.07.2026	Luca Caputi	Doku und Endkontrolle



Inhaltsverzeichnis

1. Projektidee.....	3
2. Anforderungskatalog	3
2.1 User Story 1 - Events anzeigen.....	3
2.2 User Story 2 - Event erfassen	3
2.3 User Story 3 - Event bearbeiten.....	3
2.4 User Story 4 - Event löschen.....	3
2.5 User Story 5 - Locations und Artists verwalten	4
3. Klassendiagramm	4
4. Ablauf je User Story	5
4.1 US 1 - Events anzeigen.....	5
4.2 US 2 - Event erfassen	5
4.3 US 3 - Event bearbeiten.....	5
4.4 US 4 - Event löschen.....	6
5. Datentypen je Endpoint	6
5.1 GET /events - Liste laden.....	6
5.2 GET /events/{id} - Einzelnes Event laden	6
5.3 POST /events - Event erfassen.....	6
5.4 PUT /events/{id} - Event editieren	6
5.5 DELETE /events/{id} - Event löschen.....	7
5.6 Locations und Artists.....	7
6. Testplan.....	7
7. Testdurchführung.....	7
7.1 Automatisierte Tests	8
8. Installationsanleitung.....	8
8.1 Voraussetzungen	8
8.2 Projekt holen	8
8.3 Variante A - alles mit Docker (empfohlen)	8
8.4 Variante B - Datenbank in Docker, Backend in IntelliJ	8
8.5 Kontrolle	8
9. Architektur und Aufbau	9
10. Hilfestellungen und Quellen.....	9

1. Projektidee

Mit dem Mini Event-Planer haben wir ein Backend gebaut, das die Planung von kleineren Veranstaltungen abbildet. Die Idee kam aus unserem eigenen Umfeld: Wer ein Konzert, ein Festival oder einen lokalen Anlass organisiert, muss den Überblick über drei Dinge behalten - die Veranstaltung selbst, den Ort, an dem sie stattfindet, und die Person oder Gruppe, die auftritt. Genau diese drei Bausteine bildet unsere Anwendung als Events, Locations und Artists ab. Das Backend stellt eine REST-API bereit, über die Events erfasst, abgefragt, angepasst und wieder gelöscht werden können. Jedes Event ist dabei mit einer Location und einem Artist verknüpft. Damit deckt die API den zentralen Prozess eines Veranstalters ab: Zuerst werden Orte und Künstler erfasst, danach wird darauf aufbauend ein Event geplant und bei Bedarf angepasst - etwa wenn ein Datum verschoben wird - oder gelöscht, wenn der Anlass abgesagt wird. Zielgruppe sind kleine Veranstalter bzw. Organisationsteams, die ihre Anlässe strukturiert verwalten wollen, ohne gleich eine grosse Eventmanagement-Software einzusetzen. Als Beispieldaten verwenden wir reale Anlässe aus der Schweiz und dem Ausland (z. B. den SONIC Rave in Wolfwil oder das Fliegerschiessen auf der Axalp), die beim Start der Applikation automatisch in die Datenbank geladen werden. Der Fokus des Projekts lag klar auf dem Backend: Spring Boot 3, PostgreSQL in Docker, 3-SchichtArchitektur, DTO-Mapping und automatisierte Tests. Beim Frontend haben wir nicht bei null angefangen, sondern unsere bestehende React-Applikation aus dem Modul 294 übernommen und an das neue Backend angepasst: Die Datenabfragen laufen neu über die REST-API des Spring-BootBackends, und die Datenstruktur wurde auf die neuen DTOs mit Location- und Artist-Referenzen umgestellt. Das Frontend war aber nicht Schwerpunkt dieser LB und wird in dieser Dokumentation nur am Rand erwähnt.

2. Anforderungskatalog

Die Anforderungen haben wir als User Storys mit Akzeptanzkriterien formuliert. Die Storys 1 bis 4 decken das vollständige CRUD auf der Haupt-Entity Event ab, Story 5 betrifft die verknüpften Entities Location und Artist.

2.1 User Story 1 – Events anzeigen

Als Besucher möchte ich eine Liste aller geplanten Events sehen, damit ich weiss, welche Veranstaltungen anstehen.

Akzeptanzkriterien:

- GET /events liefert Status 200 und eine JSON-Liste aller Events.
- Jedes Event enthält Titel, Datum, Beschreibung sowie die IDs der zugehörigen Location und des Artists.
- GET /events/{id} liefert ein einzelnes Event; existiert die ID nicht, kommt Status 404 zurück.

2.2 User Story 2 – Event erfassen

Als Veranstalter möchte ich ein neues Event mit Titel, Datum, Beschreibung, Location und Artist erfassen, damit der Anlass im System geplant ist.

Akzeptanzkriterien:

- POST /events legt ein Event an und liefert Status 201 Created inkl. Location-Header mit der URL des neuen Events.
- Die Antwort enthält das gespeicherte Event als DTO mit generierter ID.
- Fehlt der Titel oder ist er leer, wird das Event nicht gespeichert und die API antwortet mit Status 400 Bad Request.
- Das neue Event ist danach über GET /events/{id} abrufbar.

2.3 User Story 3 – Event bearbeiten

Als Veranstalter möchte ich ein bestehendes Event anpassen können, z. B. wenn sich das Datum oder die Location ändert.

Akzeptanzkriterien:

- PUT /events/{id} aktualisiert Titel, Datum, Beschreibung, Location und Artist und liefert Status 200 mit dem aktualisierten Event.
- Existiert die ID nicht, antwortet die API mit Status 404.
- Ein leerer Titel wird mit Status 400 abgelehnt, der bestehende Datensatz bleibt unverändert.

2.4 User Story 4 – Event löschen

Als Veranstalter möchte ich abgesagte Events löschen können, damit keine veralteten Einträge im System bleiben.

Akzeptanzkriterien:

- DELETE /events/{id} entfernt das Event und liefert Status 204 No Content.
- Existiert die ID nicht (oder wurde das Event bereits gelöscht), antwortet die API mit Status 404.
- Ein gelöscht Event ist über GET /events/{id} nicht mehr abrufbar (404).

2.5 User Story 5 – Locations und Artists verwalten

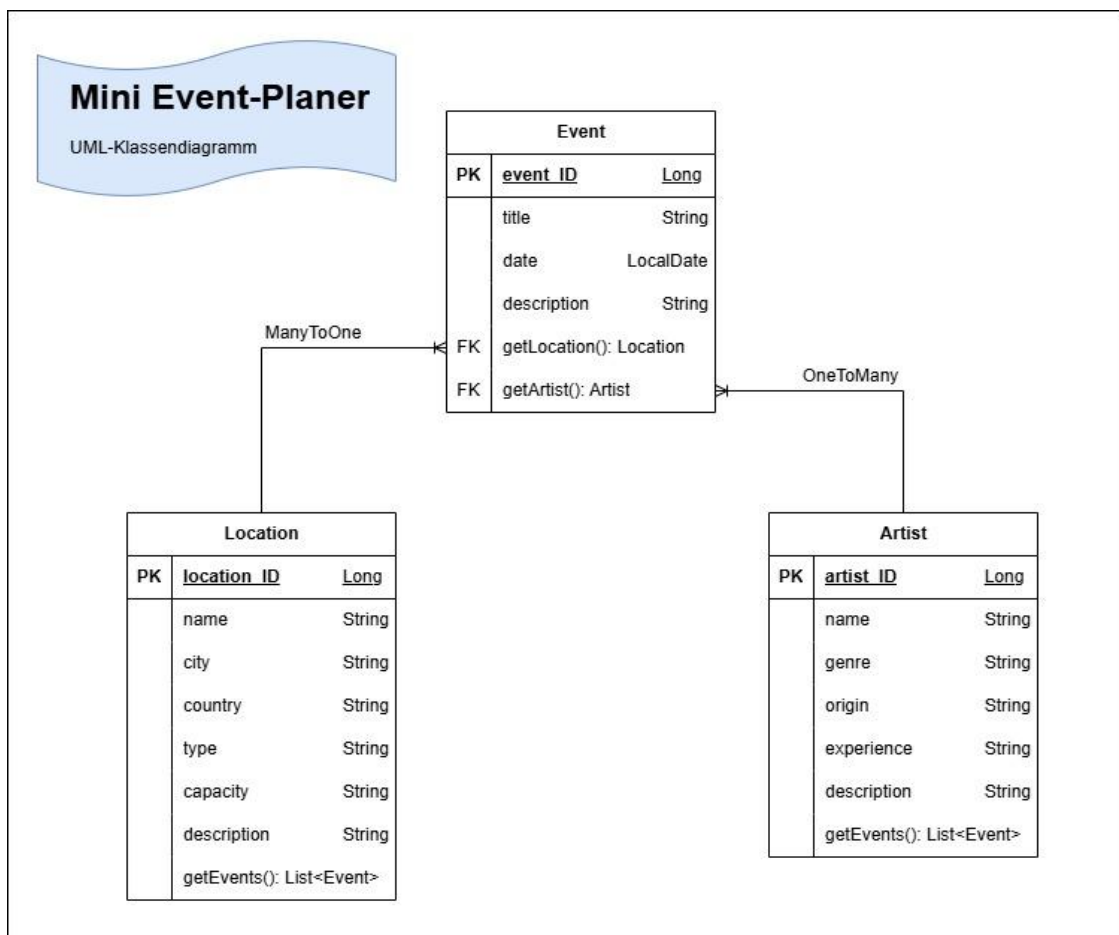
Als Veranstalter möchte ich Locations und Artists erfassen und abrufen können, damit ich sie beim Planen eines Events zuordnen kann.

Akzeptanzkriterien:

- GET /locations und GET /artists liefern jeweils Status 200 mit einer JSON-Liste (Einzelabruf per /{id} ist ebenfalls möglich).
- POST /locations bzw. POST /artists legen neue Datensätze an.
- DELETE /locations/{id} bzw. DELETE /artists/{id} entfernen einen Datensatz.

3. Klassendiagramm

Das Datenmodell besteht aus den drei Entities Event, Location und Artist. Event ist die Haupt-Entity und jeweils die Owning Side der beiden Beziehungen (@ManyToOne mit @JoinColumn auf location_id bzw. artist_id). Location und Artist bilden die Inverse Side mit @OneToMany(mappedBy = ...). Damit hat unser Modell zwei echte Entity-Beziehungen.



Beziehungsbeschreibung

Event – Artist

- Ein Event hat **genau einen Artist**
- Ein Artist kann **mehrere Events** haben
- Beziehung: **ManyToOne** (Event → Artist)
- Owning Side: **Event** (@JoinColumn(name = "artist_id"))
- Inverse Side: **Artist** (@OneToMany(mappedBy = "artist"))

Event – Location

- Ein Event findet an **genau einer Location** statt
- Eine Location kann **mehrere Events** haben
- Beziehung: **ManyToOne** (Event → Location)
- Owning Side: **Event** (@JoinColumn(name = "location_id"))
- Inverse Side: **Location** (@OneToMany(mappedBy = "location"))

Zwei Details aus der Umsetzung:

- Auf der Inverse Side (Location.events, Artist.events) verwenden wir @JsonIgnore, damit beim Serialisieren keine Endlos-Rekursion entsteht (Artist → Events → Artist → ...).
- Nach aussen geben wir nie die Entities direkt zurück, sondern DTOs (siehe Kapitel 5). Das EventDTO enthält statt der ganzen Objekte nur locationId und artistId.

4. Ablauf je User Story

Für die zentralen User Storys zeigt dieses Kapitel den Ablauf als Folge von API-Aufrufen, so wie ein Client (z. B. Insomnia oder unser Frontend) die API benutzt.

4.1 US 1 – Events anzeigen

GET /events → Liste aller Events (200) → GET /events/2 → Detail eines einzelnen Events (200).

4.2 US 2 – Event erfassen

1. GET /locations - verfügbare Locations abrufen und die gewünschte locationId auswählen.
2. GET /artists - verfügbare Artists abrufen und die artistId auswählen.
3. POST /events - neues Event mit Titel, Datum, Beschreibung, locationId und artistId anlegen → 201 Created + Location-Header.
4. GET /events/{id} - Kontrolle, dass das Event gespeichert wurde → 200.

4.3 US 3 – Event bearbeiten

1. GET /events/{id} - aktuellen Stand des Events laden → 200.
2. PUT /events/{id} - angepasstes Event-JSON senden (z. B. neues Datum) → 200 mit dem aktualisierten Event.
3. GET /events/{id} - Kontrolle, dass die Änderung persistiert wurde.

4.4 US 4 – Event löschen

1. DELETE /events/{id} → 204 No Content.
2. GET /events/{id} → 404 Not Found - das Event existiert nicht mehr.

5. Datentypen je Endpoint

Nach aussen kommuniziert die API ausschliesslich über DTOs (EventDTO, LocationDTO, ArtistDTO), die von Mapper-Klassen aus den Entities erzeugt werden. Die folgenden Beispiele zeigen echte Requests und Responses gegen die laufende Applikation (Port 8090, Daten aus dem Seeder).

5.1 GET /events – Liste laden

Response 200 OK (gekürzt auf ein Element):

```
[
  {
    "id": 2,
    "title": "Defqon 1.",
    "date": "2027-06-12",
    "description": "Festival in Holland.",
    "locationId": 2,
    "artistId": 2
  }
]
```

5.2 GET /events/{id} – Einzelnes Event laden

Request: GET /events/2 - Response 200 OK:

```
{
  "id": 2,
  "title": "Defqon 1.",
  "date": "2027-06-12",
  "description": "Festival in Holland.",
  "locationId": 2,
  "artistId": 2
}
```

Bei unbekannter ID (z. B. GET /events/999) antwortet die API mit 404 Not Found ohne Body.

5.3 POST /events – Event erfassen

Request-Body (die id wird vom Backend vergeben und deshalb weggelassen):

```
{
  "title": "Sommerfest Wolfwil",
  "date": "2026-08-15",
  "description": "Openair mit lokalen Acts.",
  "locationId": 1,
  "artistId": 1
}
```

Response 201 Created (mit Location-Header /events/5):

```
{
  "id": 5,
  "title": "Sommerfest Wolfwil",
  "date": "2026-08-15",
  "description": "Openair mit lokalen Acts.",
  "locationId": 1,
  "artistId": 1
}
```

Fehlt der Titel oder ist er leer, antwortet die API mit 400 Bad Request und speichert nichts.

5.4 PUT /events/{id} – Event editieren

Request: PUT /events/5, Body mit geändertem Datum - Response 200 OK mit dem aktualisierten Event:

```
{
  "id": 5,
```

```
{
  "title": "Sommerfest Wolfwil",
  "date": "2026-08-22",
  "description": "Openair mit lokalen Acts.",
  "locationId": 1,
  "artistId": 1
}
```

5.5 DELETE /events/{id} – Event löschen

Request: DELETE /events/5 - Response 204 No Content, ohne Body. Ein zweiter DELETE auf dieselbe ID liefert 404 Not Found.

5.6 Locations und Artists

Die beiden Neben-Entities verwenden dieselbe DTO-Logik. Beispiel-Response von GET /locations/1 bzw. GET /artists/1:

```
{
  "id": 1,
  "name": "Industrieareal Bännli",
  "city": "Wolfwil",
  "country": "Schweiz",
  "type": "Festivalgelände",
  "capacity": 15000,
  "description": "Das Industrieareal Bännli ist die Heimat der SONIC Rave Events ..."
}
{
  "id": 1,
  "name": "Ran-D",
  "genre": "Hardstyle",
  "origin": "Niederlande",
  "experience": "15 Jahre",
  "description": "Ran-D gehört zu den weltweit führenden Hardstyle-Artists ..."
}
```

6. Testplan

Die folgenden fünf manuellen Testfälle decken das CRUD auf der Haupt-Entity Event inklusive der wichtigsten Fehlerfälle ab. Durchgeführt werden sie mit Insomnia gegen die lokal laufende Applikation (<http://localhost:8090>).

Nr.	Testfall	Vorgehen (Insomnia)	Erwartetes Ergebnis
T1	Events laden	GET /events	200 OK, JSON-Liste mit den vier geseedeten Events
T2	Event erfassen (gültig)	POST /events mit gültigem JSON (Titel, Datum, locationId, artistId)	201 Created, Location-Header, Event mit neuer ID im Body
T3	Event erfassen (ungültig)	POST /events mit leerem bzw. fehlendem Titel	400 Bad Request, kein neuer Datensatz
T4	Unbekannte ID laden	GET /events/999	404 Not Found
T5	Event löschen	DELETE /events/{id} des in T2 erstellten Events, danach derselbe Aufruf nochmals	1. Aufruf: 204 No Content, 2. Aufruf: 404 Not Found

7. Testdurchführung

Alle fünf Testfälle wurden mit Insomnia gegen die laufende Applikation ausgeführt. Die Tabelle zeigt Soll und Ist im Vergleich.

Nr.	Soll	Ist	Status
T1	200 OK, Liste der Events	200 OK, vier Events aus dem Seeder werden als JSON-Liste geliefert	Bestanden

T2	201 Created + Location-Header	201 Created, Header Location: /events/5, Event mit ID im Body	Bestanden
T3	400 Bad Request	400 Bad Request, Event wird nicht gespeichert	Bestanden
T4	404 Not Found	404 Not Found, kein Body	Bestanden
T5	204, danach 404	1. DELETE: 204 No Content, 2. DELETE: 404 Not Found	Bestanden

7.1 Automatisierte Tests

Zusätzlich zu den manuellen Tests enthält das Projekt fünf automatisierte Tests, die mit mvn test ausgeführt werden. Sie laufen gegen eine H2-In-Memory-Datenbank im PostgreSQL-Modus, damit die Testsuite ohne laufenden Docker-Container funktioniert:

- EventServiceTest - Unit-Test des Service mit Mockito (gemocktes Repository, verify auf findAll).
- EventControllerIntegrationTest - GET /events liefert 200 und die gespeicherten Events (MockMvc).
- EventPostIntegrationTest - POST /events mit gültigem JSON liefert 201 und speichert das Event.
- EventErrorIntegrationTest - Fehlerfälle: GET mit unbekannter ID → 404, POST ohne Titel → 400.
- EventDeleteIntegrationTest - DELETE liefert 204, bei unbekannter ID 404.

Alle Tests laufen grün durch.

8. Installationsanleitung

8.1 Voraussetzungen

- Git
- Docker Desktop (für PostgreSQL)
- JDK 17 und IntelliJ IDEA (oder Maven auf der Kommandozeile)

8.2 Projekt holen

```
git clone https://github.com/MF-swiss/Mini_Event-Planer.git
cd Mini_Event-Planer
```

8.3 Variante A – alles mit Docker (empfohlen)

```
docker compose up --build
```

Damit werden PostgreSQL, das Backend und das Frontend zusammen gestartet. Das Backend ist danach unter <http://localhost:8090> erreichbar, das Frontend unter <http://localhost:5173>.

8.4 Variante B – Datenbank in Docker, Backend in IntelliJ

- Nur die Datenbank starten: `docker compose up -d postgres`
- Wichtig: In `docker-compose.yml` ist der Postgres-Port auf 5433 gemappt. Wird das Backend lokal (ausserhalb Docker) gestartet, muss die URL in `backend/src/main/resources/application.properties` auf `jdbc:postgresql://localhost:5433/eventdb` angepasst werden - oder das Port-Mapping in `docker-compose.yml` auf "5432:5432" geändert werden.
- Den Ordner `backend/` in IntelliJ öffnen; Maven löst alle Dependencies automatisch auf.
- Die Klasse `EventsBackendApplication` starten.

8.5 Kontrolle

Beim Start lädt der Seeder automatisch Beispieldaten (4 Events, 4 Locations, 4 Artists) in die Datenbank. Zur Kontrolle im Browser oder in Insomnia <http://localhost:8090/events> aufrufen - es muss eine JSON-Liste mit den Beispieldaten erscheinen.

Die Tests lassen sich unabhängig davon jederzeit ausführen (sie brauchen kein Docker):

```
cd backend
mvn test
```

9. Architektur und Aufbau

Das Backend folgt der klassischen 3-Schicht-Architektur. Die Controller nehmen HTTP-Requests entgegen und geben DTOs zurück, die Services enthalten die Logik, und die Repositories (Spring Data JPA) kümmern sich um den Datenbankzugriff. Kein Controller greift direkt auf die Datenbank zu.

Package	Aufgabe
controller	REST-Endpoints für /events, /locations, /artists; setzt die HTTP-Statuscodes (200, 201, 204, 400, 404)
service	Geschäftslogik, z. B. Existenzprüfung beim Löschen
repository	Datenbankzugriff über Spring Data JPA (JpaRepository)
model	JPA-Entities Event, Location, Artist inkl. Beziehungen
dto	EventDTO, LocationDTO, ArtistDTO als Java-Records - das, was die API nach aussen gibt
mapper	Konvertierung Entity und DTO in beide Richtungen
seeder	Lädt beim Start Beispieldaten aus JSON-Dateien in die Datenbank
config	CORS-Konfiguration für das Frontend

Die Validierung der Eingaben erfolgt im Controller: Ein fehlender oder leerer Titel führt bei POST und PUT zu einer 400-Antwort, unbekannte IDs werden konsequent mit 404 beantwortet, sodass die Applikation bei ungültigen Eingaben nicht abstürzt. Die Daten liegen persistent in PostgreSQL (Docker-Container mit Volume), Hibernate erzeugt das Schema über ddl-auto=update.

10. Hilfestellungen und Quellen

Folgende Hilfen haben wir während des Projekts genutzt und deklarieren sie hiermit:

- Teamarbeit: Das Projekt wurde von Luca Caputi und Marco Fritsche gemeinsam entwickelt. Wir haben die Arbeit aufgeteilt und uns gegenseitig bei Problemen unterstützt; der Stand ist über die Commit-Historie auf GitHub nachvollziehbar.
- Unterrichtsmaterial Modul 295: Das WISS-Quiz-Backend und das Webshop-Beispiel aus dem Unterricht dienten als Referenz für die 3-Schicht-Architektur, das DTO-Mapping und die Test-Typen.
- Offizielle Dokumentation: Spring Boot Reference, Spring Data JPA sowie die Baeldung-Tutorials zu @ManyToOne/@OneToMany und MockMvc.
- Stack Overflow: punktuell für einzelne Fehlermeldungen (z. B. Endlos-Rekursion bei bidirektionalen Beziehungen, gelöst mit @JsonIgnore).
- KI-Unterstützung: KI wurde punktuell als Hilfestellung bei der Fehlersuche und für die Strukturierung dieser Dokumentation eingesetzt. Der Code wurde von uns geschrieben, nachvollzogen und getestet.